Natural Deduction Rule Applier

Fatemeh Abshang

Spring 2024–2025

Abstract

This document presents a Python implementation of a natural deduction rule applier for propositional logic. The system parses formulas with line numbers, applies specified inference rules, and outputs the resulting formula. It supports eight fundamental natural deduction rules including conjunction, implication, negation operations, and modus tollens.

Contents

1 Introduction **3**
1.1 Natural Deduction . . . 3
1.1.1 Key Concepts . . . 3
1.2 Project Features . . . 3
1.3 Supported Rules . . . 3

2 System Architecture **3**
2.1 Input Format . . . 3
2.2 Data Structures . . . 4
2.2.1 Node Class . . . 4
2.2.2 Formula Storage . . . 4

3 Core Algorithms **4**
3.1 Input Parsing . . . 4
3.1.1 Formula Parsing . . . 4
3.2 Rule Application . . . 4
3.2.1 Rule Function Mapping . . . 4

4 Natural Deduction Rules **5**
4.1 AND Introduction (∧i) . . . 5
4.2 AND Elimination (∧e1, ∧e2) . . . 6
4.3 Implication Elimination (→e) - Modus Ponens . . . 6
4.4 Negation Elimination (¬e) . . . 7
4.5 Double Negation Elimination (¬¬e) . . . 8
4.6 Double Negation Introduction (¬¬i) . . . 8
4.7 Modus Tollens (MT) . . . 8

5 Tree Equality Check **9**

6 Formula Output **10**
6.1 Tree to String Conversion 10
6.2 Parenthesization Rules 10

7 Complete Examples **10**
7.1 Example 1: Modus Ponens Chain 10
7.2 Example 2: Contradiction 11
7.3 Example 3: Complex Formula 11

8 Error Handling **11**
8.1 Error Cases 11
8.2 Validation Checks 12

9 Complexity Analysis **12**
9.1 Time Complexity 12
9.2 Space Complexity 12

10 Usage Instructions **12**
10.1 Input File Format 12
10.2 Running the Program 13

11 Limitations and Extensions **13**
11.1 Current Limitations 13
11.2 Potential Extensions 13

12 Conclusion **13**

A Complete Source Code **13**

1 Introduction

1.1 Natural Deduction

Natural deduction is a proof system for propositional and predicate logic that allows derivation of conclusions from premises using inference rules that closely match human reasoning patterns.

1.1.1 Key Concepts

- **Premises**: Starting formulas (given facts)
- **Inference Rules**: Valid reasoning patterns
- **Conclusion**: Formula derived by applying rules

1.2 Project Features

- Parse formulas with line numbers from input files
- Build binary tree representations of formulas
- Apply natural deduction rules
- Validate rule applications
- Output resulting formulas

1.3 Supported Rules

Rule Name	Notation	Description
AND Introduction	$\wedge$i	Combine two formulas with AND
AND Elimination 1	$\wedge$e1	Extract left conjunct
AND Elimination 2	$\wedge$e2	Extract right conjunct
Implication Elimination	$\rightarrow$e	Modus Ponens
Negation Elimination	$\neg$e	Contradiction from A and $\neg A$
Double Negation Elim.	$\neg\neg$e	Remove double negation
Double Negation Intro.	$\neg\neg$i	Add double negation
Modus Tollens	MT	Contrapositive reasoning

Table 1: Supported natural deduction rules

2 System Architecture

2.1 Input Format

Input files contain:

1. **Formula lines**: Line number followed by 4+ spaces, then formula

2. **Rule line**: Rule name followed by comma-separated line numbers

Example input file:

```
1    p ^ q
2    q -> r
^i, 1, 2
```

2.2 Data Structures

2.2.1 Node Class

Binary tree representation of formulas:

```python
class Node:
    def __init__(self, key, left=None, right=None):
        self.data = key
        self.left: Node | None = left
        self.right: Node | None = right
```

Listing 1: Node class

2.2.2 Formula Storage

Formulas stored as dictionary mapping line numbers to tree nodes:

```python
formulas = {
    1: Node('∧', Node('p'), Node('q')),
    2: Node('→', Node('q'), Node('r'))
}
```

3 Core Algorithms

3.1 Input Parsing

3.1.1 Formula Parsing

Each formula line is split by 4+ consecutive spaces:

- Before spaces: Line number
- After spaces: Formula string

The formula string is converted to a binary tree using the parser from Phase 1.

3.2 Rule Application

3.2.1 Rule Function Mapping

Algorithm 1 Read Input File

```
function ReadInputFile
    lines ← ReadFile()
    formulaLines ← []
    ruleLine ← null
    for each line in lines do
        if line starts with digit then
            Add line to formulaLines
        else
            ruleLine ← line
        end if
    end for
    formulas ← ParseFormulas(formulaLines)
    rule ← ParseRule(ruleLine)
    return formulas, rule
end function
```

```python
RULES = {
    '∧i': AND_introduction,
    '∧e1': AND_elimination_1,
    '∧e2': AND_elimination_2,
    '→e': IMP_elimination,
    '¬e': NOT_elimination,
    '¬¬e': double_NOT_elimination,
    'MT': MT,
    '¬¬i': double_NOT_introduction,
}
```

Listing 2: Rule dispatch

4 Natural Deduction Rules

4.1 AND Introduction (∧i)

Rule:

$$\frac{A \quad B}{A \wedge B} \wedge \mathrm{i}$$

Implementation:

```python
def AND_introduction(line1, line2, formulas):
    formula1 = formulas.get(line1)
    formula2 = formulas.get(line2)

    if formula1 is None or formula2 is None:
        print("Rule Cannot Be Applied")
        return None

    return Node(AND, formula1, formula2)
```

Listing 3: AND introduction

Example:

```
Input:
1    p
2    q
^i, 1, 2

Output: p ^ q
```

4.2 AND Elimination (∧e1, ∧e2)

Rules:

$$\frac{A \wedge B}{A} \wedge \text{e1} \qquad \frac{A \wedge B}{B} \wedge \text{e2}$$

Implementation:

```python
def AND_elimination_1(line, formulas):
    formula = formulas.get(line)
    if formula is None:
        print("Rule Cannot Be Applied")
        return None

    if formula.data == AND and formula.left and formula.right:
        return formula.left
    else:
        print("Rule Cannot Be Applied")
        return None
```

Listing 4: AND elimination

Example:

```
Input:
1    p ^ q
^e1, 1

Output: p
```

4.3 Implication Elimination (→e) - Modus Ponens

Rule:

$$\frac{A \to B \quad A}{B} \to \text{e}$$

Implementation:

```python
def IMP_elimination(line_imp, line_pre, formulas):
    implication = formulas.get(line_imp)
    premise = formulas.get(line_pre)

    if implication is None or premise is None:
```

```
        print("Rule Cannot Be Applied")
        return None

    if implication.data == IMP:
        if trees_equal(implication.left, premise):
            return implication.right
        else:
            print("Rule Cannot Be Applied")
            return None
    else:
        print("Rule Cannot Be Applied")
        return None
```

Listing 5: Implication elimination

Example:

```
Input:
1    p -> q
2    p
->e, 1, 2

Output: q
```

4.4 Negation Elimination (¬e)

Rule:

$$\frac{A \quad \neg A}{\bot}\neg\mathrm{e}$$

Implementation:

```
def NOT_elimination(line1, line2, formulas):
    formula1 = formulas.get(line1)
    formula2 = formulas.get(line2)

    if formula1 is None or formula2 is None:
        print("Rule Cannot Be Applied")
        return None

    if (formula2.data == NOT and
        trees_equal(formula1, formula2.right)) or \
       (formula1.data == NOT and
        trees_equal(formula2, formula1.right)):
        return Node(FALSE)
    else:
        print("Rule Cannot Be Applied")
        return None
```

Listing 6: Negation elimination

Example:

```
Input:
```

```
1    p
2    ~p
~e, 1, 2

Output: @$\bot$@
```

4.5 Double Negation Elimination (¬¬e)

Rule:

$$\frac{\neg\neg A}{A}\neg\neg\mathrm{e}$$

Example:

```
Input:
1    ~~p
~~e, 1

Output: p
```

4.6 Double Negation Introduction (¬¬i)

Rule:

$$\frac{A}{\neg\neg A}\neg\neg\mathrm{i}$$

Example:

```
Input:
1    p
~~i, 1

Output: ~~p
```

4.7 Modus Tollens (MT)

Rule:

$$\frac{A \to B \quad \neg B}{\neg A}\mathrm{MT}$$

Implementation:

```python
def MT(line1, line2, formulas):
    formula1 = formulas.get(line1)
    formula2 = formulas.get(line2)

    if formula1 is None or formula2 is None:
        print("Rule Cannot Be Applied")
        return None

    implication = None
    neg = None

    # Check both orderings
```

```python
    if formula1.data == IMP and formula2.data == NOT:
        if trees_equal(formula1.right, formula2.right):
            implication = formula1
            neg = formula2.right
    elif formula2.data == IMP and formula1.data == NOT:
        if trees_equal(formula2.right, formula1.right):
            implication = formula2
            neg = formula1.right

    if implication and neg:
        return Node(NOT, implication.left)
    else:
        print("Rule Cannot Be Applied")
        return None
```

Listing 7: Modus Tollens

Example:

```
Input:
1    p -> q
2    ~q
MT, 1, 2

Output: ~p
```

5 Tree Equality Check

Critical utility function for comparing formula trees:

Algorithm 2 Tree Equality

```
function TreesEqual(node1, node2)
    if node1 = null and node2 = null then
        return true
    end if
    if node1 = null or node2 = null then
        return false
    end if
    if node1.data ≠ node2.data then
        return false
    end if
    return TreesEqual(node1.left, node2.left) and TreesEqual(node1.right, node2.right)
end function
```

6 Formula Output

6.1 Tree to String Conversion

Converts tree back to readable formula with proper parenthesization:

```python
def tree_to_formula_string(node):
    if node is None:
        return ""

    # Leaf node (variable or constant)
    if (node.data.isalpha() and node.data.islower())
       or node.data == FALSE:
        return node.data

    # Negation (unary)
    if node.data == NOT:
        right_str = tree_to_formula_string(node.right)
        if node.right and node.right.data in [AND, OR, IMP]:
            return f"¬({right_str})"
        return f"¬{right_str}"

    # Binary operators
    left_str = tree_to_formula_string(node.left)
    right_str = tree_to_formula_string(node.right)

    # Add parentheses based on precedence
    # [Parenthesization logic...]

    return f"{left_str} {node.data} {right_str}"
```

Listing 8: Tree to string conversion

6.2 Parenthesization Rules

- Add parentheses when child operator has lower precedence
- Special handling for right-associative implication
- Always parenthesize negation in complex expressions

7 Complete Examples

7.1 Example 1: Modus Ponens Chain

Input file:

```
1    p -> q
2    q -> r
3    p
->e, 1, 3
```

Execution:

1. Parse formulas: $p \to q$, $q \to r$, p
2. Apply →e to lines 1 and 3
3. Check: Line 1 is implication, line 3 matches antecedent
4. Result: q

Output: `q`

7.2 Example 2: Contradiction

Input file:

```
1    p ^ q
2    ~p
^e1, 1
```

Step 1 Output: `p`
Then continue:

```
1    p ^ q
2    ~p
3    p
~e, 2, 3
```

Output: $\perp$

7.3 Example 3: Complex Formula

Input file:

```
1    (p -> q) ^ (q -> r)
2    p -> q
3    q -> r
^e1, 1
```

Output: `p -> q`

8 Error Handling

8.1 Error Cases

1. **Invalid line numbers**: Referenced line doesn't exist
2. **Wrong rule application**: Formula structure doesn't match rule
3. **Missing formulas**: Required lines not provided
4. **Malformed input**: File format errors

All errors result in: `Rule Cannot Be Applied`

8.2 Validation Checks

- Check line numbers exist in formula map
- Verify formula structure matches rule requirements
- Validate tree equality for pattern matching
- Ensure correct number of arguments for each rule

9 Complexity Analysis

9.1 Time Complexity

- **Parsing**: $O(n \cdot m)$ where n is number of lines, m is formula length
- **Tree comparison**: $O(k)$ where k is tree size
- **Rule application**: $O(k)$ per rule
- **Total**: $O(n \cdot m)$

9.2 Space Complexity

- **Formula storage**: $O(n \cdot k)$
- **Result tree**: $O(k)$
- **Total**: $O(n \cdot k)$

10 Usage Instructions

10.1 Input File Format

```
[line_number]    [formula]
[line_number]    [formula]
...
[rule_name], [line1], [line2], ...
```

Requirements:

- 4+ spaces between line number and formula
- Rule line at end of file
- Comma-separated rule arguments

10.2 Running the Program

```
python natural_deduction.py

please enter a file name:
proof.txt

# Output: resulting formula or error message
```

Listing 9: Execution

11 Limitations and Extensions

11.1 Current Limitations

1. Only 8 basic rules supported
2. No scope/assumption handling
3. Single rule application per run
4. No proof validation

11.2 Potential Extensions

1. Add OR introduction/elimination
2. Support implication introduction
3. Implement full proof checker
4. Add interactive mode
5. Support proof strategies

12 Conclusion

This natural deduction rule applier provides a foundation for automated reasoning in propositional logic. The system correctly parses formulas, applies inference rules, and validates applications. While limited to basic rules, it demonstrates core concepts that extend to complete natural deduction systems.

A Complete Source Code

```
import sys

# Define logical symbols
NOT = '¬'
AND = '∧'
```

```python
OR = '∨'
IMP = '→'
FALSE = '⊥'

OPERATOR_PRECEDENCE = {
    IMP: 1, OR: 2, AND: 3, NOT: 4
}

class Node:
    def __init__(self, key, left=None, right=None):
        self.data = key
        self.left: Node | None = left
        self.right: Node | None = right

def read_input_file():
    print("please enter a file name:")
    file_name = input()

    with open(file_name, 'r', encoding='utf-8') as file:
        lines = [line.strip() for line in file if line.strip()]

    formula_lines = []
    rule_line = None

    for line in lines:
        if line and line[0].isdigit():
            formula_lines.append(line)
        else:
            rule_line = line

    formulas = parse_formulas_from_lines(formula_lines)
    rule = read_rule(rule_line) if rule_line else None

    return formulas, rule

def parse_formulas_from_lines(lines):
    line_formula = {}
    for line in lines:
        if '    ' not in line:
            continue

        line_number_str, formula_str = line.split('    ', maxsplit=1)
        formula_str = formula_str.replace(" ", "")

        line_number = int(line_number_str)
        formula_tree = binary_tree(formula_str, 0, len(formula_str)-1)
        if formula_tree is None:
            continue
        line_formula[line_number] = formula_tree
```

```python
    return line_formula

def read_rule(line):
    if not line:
        return None
    parts = line.split(',')
    rule_name = parts[0].strip()
    rule_lines = [int(p.strip()) for p in parts[1:]]
    return rule_name, rule_lines

# Rule implementations
def AND_introduction(line1, line2, formulas):
    formula1 = formulas.get(line1)
    formula2 = formulas.get(line2)
    if formula1 is None or formula2 is None:
        print("Rule Cannot Be Applied")
        return None
    return Node(AND, formula1, formula2)

def AND_elimination_1(line, formulas):
    formula1 = formulas.get(line)
    if formula1 is None:
        print("Rule Cannot Be Applied")
        return None
    if formula1.data == AND and formula1.left and formula1.right:
        return formula1.left
    else:
        print("Rule Cannot Be Applied")
        return None

def IMP_elimination(line_imp, line_pre, formulas):
    implication = formulas.get(line_imp)
    premise = formulas.get(line_pre)
    if implication is None or premise is None:
        print("Rule Cannot Be Applied")
        return None
    if implication.data == IMP:
        if trees_equal(implication.left, premise):
            return implication.right
        else:
            print("Rule Cannot Be Applied")
            return None
    else:
        print("Rule Cannot Be Applied")
        return None

def trees_equal(node1, node2):
    if node1 is None and node2 is None:
        return True
    if node1 is None or node2 is None:
        return False
```

```python
    if node1.data != node2.data:
        return False
    return (trees_equal(node1.left, node2.left) and
            trees_equal(node1.right, node2.right))

RULES = {
    '∧i': AND_introduction,
    '∧e1': AND_elimination_1,
    '∧e2': AND_elimination_2,
    '→e': IMP_elimination,
    '¬e': NOT_elimination,
    '¬¬e': double_NOT_elimination,
    'MT': MT,
    '¬¬i': double_NOT_introduction,
}

def apply_rule(rule_name, lines_to_use, formulas):
    rule_function = RULES.get(rule_name)
    if rule_function is None:
        print("Rule Cannot Be Applied")
        return

    result_node = None
    if rule_name in ['∧i', '¬e', 'MT', '→e']:
        if len(lines_to_use) != 2:
            print("Rule Cannot Be Applied")
            return
        result_node = rule_function(lines_to_use[0],
                                    lines_to_use[1], formulas)
    elif rule_name in ['∧e1', '∧e2',
                       '¬¬e', '¬¬i']:
        if len(lines_to_use) != 1:
            print("Rule Cannot Be Applied")
            return
        result_node = rule_function(lines_to_use[0], formulas)
    else:
        print("Rule Cannot Be Applied")
        return

    if result_node:
        print(tree_to_formula_string(result_node))

def main():
    formulas_map, rule_data = read_input_file()
    if rule_data:
        rule_name, lines_to_use = rule_data
        apply_rule(rule_name, lines_to_use, formulas_map)
    else:
        print("Error: Rule information not found in input file.")

if __name__ == "__main__":
```

```
    main()
```

Listing 10: Natural Deduction Rule Applier